

RAPID Oni Operations Manual

This manual describes the RAPID Community Data Lab (RAPID-CDL) deployment of an Oni-based archival repository. It is for:

- system administrators evaluating or operating the RAPID Oni stack;
- developers integrating with the API or preparing RO-Crate data for deposit;
- maintainers responsible for deployments, indexing, backups, and recovery.

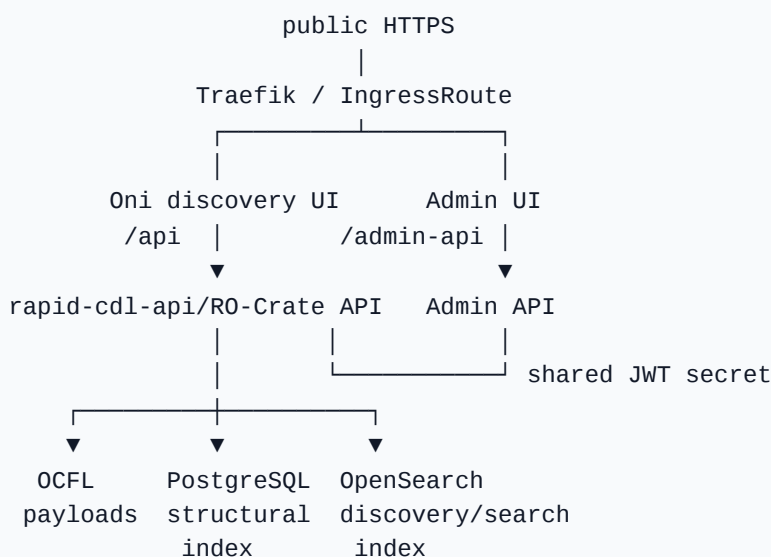
The scope is the stack as deployed by this project. It does not describe alternative Oni architectures.

Commands use the local Docker Compose setup unless explicitly marked Kubernetes.
Replace example hostnames and namespaces for each environment.

1. Overview of the RAPID Oni stack

1.1 Architecture

The stack separates durable repository data from the services that expose and index it:



The Kubernetes `rcdl` manifests deploy the API, Oni UI, admin UI, admin API, PostgreSQL, an OpenSearch service pointing to a dedicated OpenSearch VM, Traefik routes, TLS certificates, persistent storage, and sealed secrets. The local Compose projects use the external `rapid-community-data-lab` Docker network.

Request	Local Docker	Kubernetes role
Discovery portal	<code>http://localhost:8081</code>	Oni UI at the data host root

Request	Local Docker	Kubernetes role
Public API	<code>http://localhost:8080</code>	<code>/api</code> , with prefix stripped
Admin portal	<code>http://localhost:8082</code>	Admin UI at the admin host root
Admin API	<code>http://localhost:8083</code>	<code>/admin-api</code> , with prefix stripped

Kubernetes ingress uses `data.<RCDL_FQDN>` for the portal and `admin.<RCDL_FQDN>` for the admin application. The public FQDN is supplied through cluster variables.

1.2 Main components

Oni

Oni is the discovery portal: a Vue application served by nginx. Nginx proxies `/api/*` to the backend specified by `BACKEND_URL` . The image is therefore environment-agnostic. Runtime `configuration.json` controls branding, navigation, search, facets, maps, metadata display, terms, privacy, and help. Docker mounts this file; Kubernetes supplies it with a ConfigMap.

OCFL

OCFL (Oxford Common File Layout) is the durable representation of repository objects. An object contains inventories, versioned content, and fixity data. Content directories hold crate files; inventory JSON describes versions and content. OCFL is the authoritative source for deposited RO-Crate metadata and payload files.

RAPID uses OCFL 1.1, the direct-storage layout, and CRC32 fixity:

Setting	Docker API container	Kubernetes API container
<code>OCFL_PATH</code>	<code>/data/ocfl/data</code>	<code>/data/ocfl/data</code>
<code>OCFL_SCRATCH</code>	<code>/data/ocfl/scratch</code>	<code>/data/ocfl/scratch</code>

The local stack bind-mounts `rapid-community-data-lab-api/.ocfl` . Kubernetes mounts the `ocfl-data` ReadWriteOnce PVC at `/data/ocfl` . Scratch space is not a backup.

RAPID Community Data Lab API (rapid-cdl-api)

The deployed API is the RAPID implementation of Arocap, an RO-Crate API built with Fastify, TypeScript, Prisma, PostgreSQL, and OpenSearch. In this manual, “rapid-cdl-api” refers to this API layer and its standard RO-Crate routes.

It provides entity and file retrieval, RO-Crate metadata retrieval, listing, and OpenSearch-backed search. It serves files from OCFL, uses PostgreSQL for the structural index, and uses OpenSearch for discovery/search records. The `/version` endpoint is the primary basic health check.

Its main route groups are:

- public RO-Crate API routes such as `GET /entities`, `GET /entity/:id`, `GET /entity/:id/rocrate`, `GET /files`, `GET /file/:id`, and `POST /search`;
- `GET /version` for a basic health/version check;
- authenticated repository-management routes under `/admin`, including `GET /admin/repository`, `GET /admin/index/:crateId?`, `POST /admin/index` or `POST /admin/index/:crateId/:type?`, and `DELETE /admin/index` or `DELETE /admin/index/:crateId/:type?`.

The indexing routes operate on derived PostgreSQL and OpenSearch indexes. They do not upload or delete OCFL payloads. When accessed through the Kubernetes or local Traefik `/api` route, the external paths include the `/api` prefix (for example, `/api/admin/index`); the proxy strips that prefix before forwarding the request to the API.

The current code uses `AllPublicAccessTransformer` and `AllPublicFileAccessTransformer`. Repository content is public as deployed; admin login does not make portal content private.

Admin UI

The admin UI is a Vue application served as static files by Caddy. It can log in, list OCFL repository objects, request indexing for all or selected objects/subtrees, and request deletion of derived indexes. Its browser local-storage index state is only a display convenience; verify operations with API responses, counts, and logs.

Admin API

The admin API stores administrator accounts in PostgreSQL's `admin_users` table and exposes:

- `POST /login`, returning a JWT valid for one hour;
- `POST /create-admin`, for an authenticated administrator;
- `GET /health`, returning `{"status":"ok"}`.

These are authentication and service-health routes only. They are separate from the repository API's authenticated `/admin/repository` and `/admin/index/*` routes described above. The admin UI calls the Admin API for login, then sends the returned bearer token to the repository API when it lists objects or starts/deletes indexing.

The admin API and repository API must use exactly the same `API_AUTH_JWT_SECRET`. The repository API accepts only JWTs with the `ADMIN` or `SUPER_ADMIN` role on `/admin/*`.

2. Setting up a development environment

2.1 Prerequisites

- Docker 24+ and Docker Compose v2;

- Node.js 24+ for the RAPID API;
- pnpm for Oni UI development;
- the three repositories checked out as siblings: `rapid-community-data-lab-api`, `oni-ui`, and `rapid-cdl-admin`.

Never commit `.env`, credentials, access tokens, or production configuration.

2.2 Start the local backend

From `rapid-community-data-lab-api`:

```
cp .env.example .env
docker network create rapid-community-data-lab
docker compose up -d
curl -s https://data.rapid-cdl.edu.au/api/version
```

Set a strong `API_AUTH_JWT_SECRET`; use the same value for the admin API. Compose starts PostgreSQL (`db`), OpenSearch (`opensearch`), and the API (`api`). The API entrypoint waits for both data services, synchronises the Prisma schema, and starts Fastify.

For host-side API development:

```
docker compose up -d db opensearch
npm install
npm run db:sync
npm run dev
```

Main API settings are `DATABASE_URL`, `OPENSEARCH_URL`, `API_AUTH_JWT_SECRET`, `RAPID_COMMUNITY_DATA_LAB_API_PORT`, `OCFL_PATH`, `OCFL_SCRATCH`, and `LOG_LEVEL`.

2.3 Start Oni UI

For browser development:

```
cd oni-ui
cp configuration.sample.json configuration.json
pnpm run setup:vocabs vocab.json
cd src
pnpm install
pnpm run dev
```

Open `http://localhost:5173`. Configure the API endpoint as `/api` when using the nginx proxy. For running with Docker (production-like) container:

```
docker compose -f docker/docker-compose.yml up -d --build
curl -s https://data.rapid-cdl.edu.au/api/version
```

The container mounts `configuration.json` read-only and uses `BACKEND_URL=http://api:8080` on the shared network.

2.4 Start admin UI and admin API

Set `DATABASE_URL`, `PORT`, and the shared `API_AUTH_JWT_SECRET` in `rapid-cdl-admin/api/.env` (or supply them through Compose), then:

```
cd rapid-cdl-admin/api
docker compose up -d --build
cd ..
docker compose --env-file .env -f docker/docker-compose.yml up -d --build
curl -s https://admin.rapid-cdl.edu.au/api/version
```

The local admin API is on port 8083 and the admin UI/Traefik edge is on 8082. Create the first administrator using the project bootstrap procedure. Do not put administrator passwords in shell history or source files.

2.5 Load and index local test data

The bundled corpus is for development, not a production deposit. After obtaining a JWT from `POST /login`:

```
export ADMIN_JWT='<token returned by /login>'
curl -L -X POST -H "Authorization: Bearer ${ADMIN_JWT}" \
  https://data.rapid-cdl.edu.au/api/admin/index/
```

Indexing is asynchronous:

```
docker compose logs -f api | grep -E 'Indexing|level":50'
curl -s 'https://data.rapid-cdl.edu.au/api/entities?limit=1'
```

Do not treat `202 Accepted` alone as completion; wait for logs and verify a non-zero entity total.

3. Deployment

3.1 Data storage

The local API stack starts three containers attached to the shared network:

Service	Container name	Hostname	Host port
PostgreSQL	<code>db</code>	<code>db</code>	5432
OpenSearch	<code>opensearch</code>	<code>opensearch</code>	9200, 9300
API	<code>api</code>	<code>api</code>	8080

The storage-related runtime settings are:

Variable	Default	Notes
<code>DATABASE_URL</code>	(required)	PostgreSQL connection string.
<code>OPENSEARCH_URL</code>	<code>http://localhost:9200</code>	OpenSearch endpoint.
<code>OCFL_PATH</code>	<code>./ocfl/data</code>	OCFL repository root.
<code>OCFL_SCRATCH</code>	<code>./ocfl/scratch</code>	OCFL working directory.

For Kubernetes, create a Deployment and Service exposing port `8080` , then inject `DATABASE_URL` , `OPENSEARCH_URL` , `API_AUTH_JWT_SECRET` , and related settings through Secrets or ConfigMaps.

The frontend is stateless. Local persistent test data belongs to the backend stack:

Data	Local storage	Removal
PostgreSQL rows	named volume <code>rapid-community-data-lab-api_postgres_data</code>	<code>docker compose down -v</code>
OpenSearch index	named volume <code>rapid-community-data-lab-api_opensearch_data</code>	<code>docker compose down -v</code>
OCFL repository	bind mount <code>./rapid-community-data-lab-api/ocfl</code>	restore from git or regenerate

Normal `docker compose down` keeps these local data stores for reuse. The admin UI uses the same backend stack; its static frontend does not own repository storage.

3.2 Deploying components

The deployment is environment-variable driven and listens on port `8080` . The deployment actions are:

- build and push the API image from its `Dockerfile` in the GitLab CI pipeline;
- create a `Deployment` and matching `Service` for port `8080` ;
- inject `DATABASE_URL` , `OPENSEARCH_URL` , `API_AUTH_JWT_SECRET` , and other settings through Kubernetes `Secret` or `ConfigMap` values;
- set the Oni UI `BACKEND_URL` to the in-cluster API Service URL.

The Oni UI image is environment-agnostic. Its runtime setting is:

Variable	Default	Purpose
<code>BACKEND_URL</code>	<code>http://api:8080</code>	Upstream URL that nginx proxies <code>/api/*</code> to.

Changing `BACKEND_URL` takes effect by recreating the pod; the image does not need to be rebuilt.

The admin UI is a static Caddy-served SPA. In the local and Kubernetes setups, Traefik routes `/api/*` to the backend and the remaining paths to the admin UI.

3.3 Configuration and secrets

Keep these values aligned:

- `API_AUTH_JWT_SECRET` in both APIs;
- API `DATABASE_URL` and the PostgreSQL cluster/service;
- API `OPENSEARCH_URL` and the OpenSearch service/endpoint;
- Oni `BACKEND_URL` and the API Service name/port;
- portal `configuration.json` and the public API base path.

Kubernetes stores sensitive values in SealedSecrets. Back up the encrypted manifests and the Sealed Secrets controller recovery material according to cluster policy. A SealedSecret manifest alone cannot recover plaintext secrets on a different cluster.

3.4 Access control

The admin API issues the bearer token used by the repository API. Both APIs must use the same `API_AUTH_JWT_SECRET`. The admin UI calls authentication through the same-origin `/admin-api` path, which is routed to the admin API by local Traefik and production Kubernetes.

3.5 Security

The Oni Docker setup identifies the following configuration items that must be customised:

- OAuth credentials, including GitHub or CILogon authentication;
- all security tokens and secrets;
- API database settings, `API_AUTH_JWT_SECRET`, and `OPENSEARCH_JAVA_OPTIONS`.

3.6 Monitoring

For the deployed admin application, use the public HTTPS host. The repository API and admin API are routed through this host; port `8083` is not accessed directly from outside the cluster:

```
# Repository API through the production ingress
curl -s https://data.rapid-cdl.edu.au/api/version

# Admin API through the production ingress
curl -s https://admin.rapid-cdl.edu.au/api/version

# Admin UI; follow the unauthenticated redirect to /login
curl -sS -L -o /dev/null \
  -w '%{http_code} %{url_effective}\n' \
  https://admin.rapid-cdl.edu.au/
```

The final command should finish at `https://admin.rapid-cdl.edu.au/login`.

Indexing runs asynchronously. Follow progress and check the entity count with:

```
docker compose logs -f api | grep -E 'Indexing|level":50'
curl -s 'https://data.rapid-cdl.edu.au/api/entities?limit=1'
```

4. Upgrading components

The Oni UI image supports these tags:

- `latest` : latest build from the main branch;
- `vX.Y.Z` : specific version releases;
- `main` : latest build from main;
- `sha-xxxxxx` : specific commit SHA.

After pushing to the `main` branch, check GitHub Actions, Docker Hub, and GitHub Container Registry to confirm that the image was published.

Build and push the RAPID API image from its `Dockerfile` in the GitLab CI pipeline.

5. Onboarding and managing data collections

5.1 Process summary

The current local workflow covers the bundled test data and indexing.

5.2 Adding data

For the bundled local test data, log in through `rapid-cdl-admin`, use the returned JWT as the bearer token, and configure the same `API_AUTH_JWT_SECRET` in both APIs:

```
export ADMIN_JWT='<token returned by /login>'
curl -L -X POST -H "Authorization: Bearer ${ADMIN_JWT}" \
  http://localhost:8080/admin/index/
```

Indexing runs asynchronously. Follow progress and check the resulting entity count with:

```
docker compose logs -f api | grep -E 'Indexing|level":50'
curl -s 'https://data.rapid-cdl.edu.au/api/entities?limit=1'
```

5.3 Deleting data

The clean-slate procedure is destructive and local only. It permanently deletes the local PostgreSQL database, OpenSearch index, and OCFL data written at runtime. It only touches local Docker volumes and the local `../rapid-community-data-lab-api/.ocfl` working directory, and must never be run against a shared or remote environment.

5.4 Indexing operations

The admin UI calls authentication through the same-origin `/admin-api` path. Local Traefik and production Kubernetes route this path to the admin API. The indexing request is sent to the repository API at `/admin/index/`. For a specific crate, use `POST /admin/index/:crateId`; through the production `/api` route, this becomes `POST /api/admin/index/:crateId`.

The indexing flow is:

- **OCFL import:** OCFL is the system of record. The crate is streamed into `/data/ocfl/staging`, then `ocfl-fs` writes it into the OCFL root with `object.import()`.
- **Index request:** `POST /admin/index/:crateId` returns `202` and queues the work in a `PromiseQueue` with concurrency four.
- **Two indexers:** the crate is parsed once, then sent to:
 - `StructuralIndexer` → PostgreSQL, using Prisma upserts for `entity` and `file` records;
 - `SearchIndexer` → OpenSearch, using bulk indexing for entity records.
- **Serving data:** PostgreSQL serves `/entities` and `/entity/:id`, OpenSearch serves `/search`, and OCFL streams the deposited bytes and files.

6. Backup and recovery

CloudNativePG is used to natively online/hot backup of PostgreSQL clusters through continuous physical backup and WAL archiving. The database is always up (no downtime required) and it recovers at any point in time from the first available base backup in the system.

7. Monitoring and troubleshooting

7.1 Basic checks

For the production deployment, first check the public HTTPS endpoints:

```
curl -s https://data.rapid-cdl.edu.au/api/version
curl -s https://admin.rapid-cdl.edu.au/api/version
curl -s -L -o /dev/null \
  -w '%{http_code} %{url_effective}\n' \
  https://admin.rapid-cdl.edu.au/
```

The browser-facing root URL should redirect unauthenticated users to `https://admin.rapid-cdl.edu.au/login`.

The Kubernetes workloads run in the `rcdl` namespace. Use `k9s -n rcdl` for interactive cluster monitoring. Use `kubectl` for repeatable checks and for collecting details when troubleshooting:

```
# List workloads, services, and pod placement
kubectl get pods -n rcdl -o wide

# Check rollout
kubectl rollout status deployment/rapid-community-data-lab-api -n rcdl
kubectl rollout status deployment/rapid-cdl-admin-api -n rcdl

# View recent application logs
kubectl logs -n rcdl deployment/rapid-community-data-lab-api \
  -c api --tail=100
kubectl logs -n rcdl deployment/rapid-cdl-admin-api \
  -c admin-api --tail=100

# Follow API logs while reproducing a problem
kubectl logs -n rcdl deployment/rapid-community-data-lab-api \
  -c api --follow

# View logs from the previous container after a restart
kubectl logs -n rcdl deployment/rapid-community-data-lab-api \
  -c api --previous --tail=100

# Inspect a specific pod and its recent events
kubectl describe pod -n rcdl <pod-name>
```

The main workloads are `rapid-community-data-lab-api`, `rapid-cdl-admin-api`, `rapid-cdl-admin`, and `oni-ui`. A pod that is not Ready, repeated restarts, failed probes, or image-pull errors should be investigated with `kubectl describe pod` and the relevant `kubectl logs` command.

7.2 Common problems

For indexing, use this check:

```
curl -s 'https://data.rapid-cdl.edu.au/api/entities?limit=1'
```

Indexing arcp://... lines should appear for each bundled crate.

7.3 Local clean-slate reset

The clean-slate procedure is destructive and local only. `docker compose down -v` removes the `postgres_data` and `opensearch_data` volumes, wiping the database and search index. The procedure also resets the local `.ocf1` directory.